

Deep Learning - lab 1

An introduction to Python

Prof. Stefano Carrazza

University of Milan and INFN Milan

Introducing Python

Introducing Python

Python	C++
Paradigm Multi-paradigm (object-oriented, procedural, functional, declarative, reflective) Designed by Guido van Rossum Developer Python Software Foundation First appeared 20 January 1989, 30 years ago Stable release 3.12.0 (7 Feb 2024) 36 days ago Previous release 3.11.9 (12 Oct 2023) 14 days ago Typing discipline None, dynamic (static typing) OS cross-platform License Python Software Foundation License Platform extensions yes, yes, yes, and yes (later to C, C#, Java, etc.) Website python.org Major implementations CPython, PyPy, Jython, IronPython, MicroPython, CPython, PyPy, etc. Stability Python Software Foundation Influenced by ABC, Modula-2, Pascal, C, C++, Perl, Ruby, etc. Influenced Raku, Perl 6, etc.	Paradigm Single-paradigm (Standard C++) Designed by Bjarne Stroustrup Developer Bjarne Stroustrup, AT&T Bell Laboratories, etc. First appeared 1985, 39 years ago Stable release 14.0.0 (2023) 13 months ago Previous release 13.0.0 (2022) 15 months ago Typing discipline static, non-strictly-typed OS cross-platform Platform extensions yes, yes, yes, and yes (later to C, C#, Java, etc.) Website en.cppreference.com Major implementations GCC, Clang, Microsoft Visual C++, Intel C++ Compiler, etc. Influenced by C, C#, Java, etc. Influenced C#, Java, etc.

Some famous Python libraries:



Most popular programming language



Python is the preferred programming language for ML applications, furthermore there are several modules which simplifies data analysis tasks.

2

Most popular public DL frameworks

Some frameworks for deep learning deployment:

- Keras: a Python deep learning library.
- TensorFlow: library for numerical computation with data flow graphs.
- PyTorch: a DL framework for fast, flexible experimentation.
- Caffe: speed oriented deep learning framework.
- Theano: a Python library for optimization.
- MXNet: deep learning framework for neural networks.
- CNTK: Microsoft Cognitive Toolkit.
- scikit-learn: general machine learning package.

paghe face book

riao left

Why use public codes? → builtin models and automatic differentiation

3

How to install python?

Python comes pre-installed in UNIX-based systems.

Anyway there are official binaries for all platforms in <https://www.python.org>.

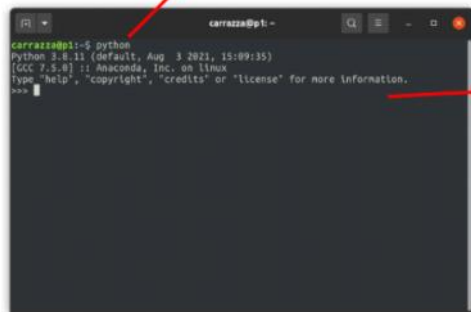


Warning: python3 != python2. Use python3.

4

How to install python?

Launch python command.



```
carranza@pt: ~  
carranza@pt:~$ python  
Python 3.8.11 (default, Aug 3 2021, 15:09:35)  
[GCC 7.5.0] on Linux  
Type "help", "copyright", "credits" or "license()" for more information.  
>>>
```

Type: exit() or CTRL+D

5

Anaconda

Anaconda and miniconda are other simple alternatives for a full integrated development environment: <https://anaconda.org>

Data science technology for a better world.

A movement that brings together millions of data science practitioners, data-driven enterprises, and the open source community.



6

Jupyter notebooks

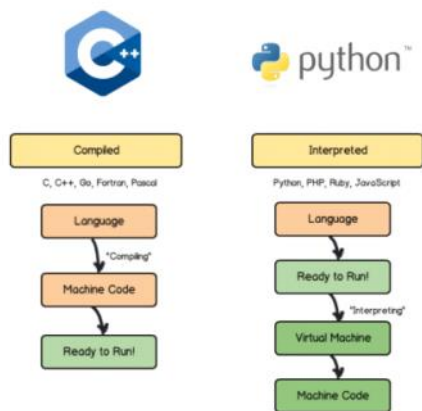
You could even code directly in your browser thanks to Jupyter Notebooks, and services like Google Colab: <https://colab.research.google.com>



7

The Python interpreter

The Python interpreter



8

The Python interpreter



Pro

- Fast prototyping
- Large community, huge number of libraries

Cons

- Bad performance for intensive computation using native python primitives.
- Installation process and dependency resolver not trivial.

9

The Python modules

Third-party libraries can be imported easily in your code:



Numerical Python
<https://numpy.org/>

Visualization with Python
<https://matplotlib.org/>

Example:

```
pip install numpy
pip install matplotlib
```

10

The Python language

The Python language

Printing to terminal:

```
print("Hello World!")
```

Define a script `myscript.py`:

```
#!/usr/bin/env python
print("Hello World!")
```

Or "chmod +x script.py" and then "python myscript.py".

11

The Python language

Variable allocation (dynamic typing):

```
a = 1          # integer
b = -16.5      # double
c = "Hello"    # string
d = True       # bool
e = 1+5j       # complex
```

Printing:

```
print(a, b, c, d, e)
print(f'{a} {b} {d} {e}') # f-string
```

12

The Python language

Lists:

```
e = []          # empty list
f = [1, 2]       # initialized list

f.append(5)      # append to list
f += [6]         # append to list

list_length = len(f) # returns 4
```

13

The Python language

Dictionaries:

```
g = {'key1': 5,
     'key2': [1, 2]}

# getter
v = g.get('key1')

# setter
g['key3'] = 6
```

14

The Python language

Conditionals:

```
if condition1:
    <code>
elif condition2:
    <code>
else:
    <code>
```

Example:

```
a = input("positive integer:")
test = a > 0
if test:
    print("input is positive")
else:
    print("input is negative")
```

15

The Python language

Control flow statement for:

```
n = 10
for i in range(n):
    <code>
    break
    continue
<code>
```

Control flow statement while:

```
condition = True
while condition:
    <code>
    break
    continue
<code>
```

16

The Python language

Functions:

```
def myfunction(arguments):
    r = <code>
    return r
```

Main-like script:

```
def myfunction(x, p):
    return x**p

if __name__ == "__main__":
    v = myfunction(5, 2)
    print(v)
```

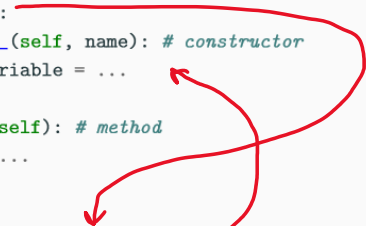
17

The Python language

Classes:

```
class MyClass:
    def __init__(self, name): # constructor
        self.variable = ...

    def sample(self): # method
        return ...
```

Two red arrows originate from the `MyClass` definition. One arrow points from the `__init__` method to the `super().__init__()` call in the `ChildClass` constructor. The other arrow points from the `sample` method to the `ChildClass` class definition, indicating that `ChildClass` inherits from `MyClass`.

Inheritance:

```
class ChildClass(MyClass):
    def __init__(self):
        super().__init__() # executes MyClass constructor
        self.variable *= 2
```

18

Tutte le eredità sono pubbliche

The Python language

```
import numpy as np

def myfunction(x):
    """Evaluates exponential of x^2"""
    return np.exp(-np.square(x))

def main():
    """Main function demo."""
    a = 5          # int
    b = "hello"    # string
    c = 6j          # complex

    results = []
    for ix in range(10):
        results.append(myfunction(ix))

    if results[0] != 1:
        print("Error")

if __name__ == "__main__":
    main()
```

19

Deep Learning - lab 2

An introduction to Numpy, Matplotlib, Scipy and TensorFlow

Prof. Stefano Carrazza

University of Milan and INFN Milan

Librerie

Installing libraries

Third-party libraries can be imported easily in your code:



Numerical Python
<https://numpy.org/>

Visualization with Python
<https://matplotlib.org/>

Example:

```
pip install numpy matplotlib pandas scipy tensorflow
# Or
conda install numpy matplotlib pandas scipy tensorflow
```

1

Numpy basics

Arrays

Let's suppose we have the matrix (32bit floats):

$$a = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \\ -1 & 0 & 3 \end{pmatrix}$$

We allocate this matrix with:

```
import numpy as np
a = np.array([[1, 0, 0], [0, 1, 2], [-1, 0, 3]], dtype=np.float32)

type(a) # <class 'numpy.ndarray'>
a.shape # returns (3,3)
a.ndim # 2
a.dtype # dtype('float32')
a.flatten() # [1, 0, 0, 0, 1, 2, -1, 0, 3]
```

Numpy provides several dtypes: int32, int64, complex32, complex64, float32, float64...

2

Arrays

```
import numpy as np

# building array with zeros:
z = np.zeros(5, dtype=np.int32)
# array([0, 0, 0, 0, 0], dtype=int32)

# building array with ones
o = np.ones((5,1), dtype=np.float64)

# reshaping array
w = o.reshape(1, 5)
# array([[1., 1., 1., 1., 1.]])
```

3

Math operations

```
import numpy as np
a = np.array([1, 2, 3, 5])
b = np.ones(4)

c = a + b
# array([2., 3., 4., 6.])

d = c ** 2
e = 2 * np.sin(d)
# array([-1.51360499,  0.82423697, -0.57580663, -1.98355771])

e > 0
# array([False,  True, False, False])
```

4

Linear algebra

```
import numpy as np

A = np.eye(2) # [[1, 0], [0, 1]]
B = np.array([[2, 1], [3, 4]])

# elementwise product
A * B # [[2, 0], [0, 4]]

# matrix product
A @ B # or A.dot(B)
# [[2, 1], [3, 4]]

# transpose matrix
B.T
# [[2, 3], [1, 4]]
```

5

Slicing

```
import numpy as np

C = np.array([[2, 1], [3, 4]])

m = 0
C[m, m] # 2
C[:, m] # [2, 3]
C[m, :] # [2, 1]
C[m, -1] # 1
C[-1, m] # 3

C.flatten()[0] # 2
C.flatten()[-1] # 4
C.flatten()[0:2] # [2, 1]
```

6

Copy

```
import numpy as np

a = np.array([[2, 1], [3, 4]])

# no copy
b = a # a and b are the same ndarray

# shallow copy
c = a.view()
c[0, 0] = -1 # a's data change
c = c.reshape(4) # a's shape doesn't change

# deep copy
d = a.copy()
```

7

Input/Output

```
import numpy as np

# store array
a = np.array([[2, 1], [3, 4]])
np.save("myarray.npy", a) # binary format
np.savetxt("myarray.txt", a) # or text format

# read from file
b = np.load("myarray.npy")
b = np.loadtxt("myarray.txt")
```

8

hdf5 ~~sto~~ Libreria
per salvare roba

Matplotlib basics

Anche seaborn è buono, più dedicato ad analisi statistiche

d3js->d3blocks è anche un'altra libreria molto buona, correlazioni, mappe, schematici.
Esiste un modulo in python per gestirlo

Basics

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2, 100) # 100 points linearly spaced [0,2]

plt.plot(x, x**2, label="quadratic")

plt.title("Quadratic curve")
plt.xlabel("x label")
plt.ylabel("y label")
plt.legend()

plt.savefig("myplot.png")
plt.show() # keeps the plot open if script
```

9

Basics

https://matplotlib.org/cheatsheets/_images/cheatsheets-1.png

10

Pandas basics

Series

```
import pandas as pd
import numpy as np

s = pd.Series([1, 3, 5, np.nan, 6, 8])
print(s)
# 0    1.0
# 1    3.0
# 2    5.0
# 3    NaN
# 4    6.0
# 5    8.0
# dtype: float64
```

11

Dataframes

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "A": 1.0,
    "B": pd.Series(1, index=list(range(4)), dtype="float32"),
    "C": np.array([3] * 4, dtype="int32"),
    "D": pd.Categorical(["test", "train", "test", "train"]),
    "E": "foo",})

# A    B    C    D    E
# 0  1.0  1.0  3  test  foo
# 1  1.0  1.0  3  train  foo
# 2  1.0  1.0  3  test  foo
# 3  1.0  1.0  3  train  foo
```

12

Viewing table

```
df.head() # head of the table
df.tail(3) # tail (last 3 lines) of the table

df.index
# Int64Index([0, 1, 2, 3], dtype='int64')

df.columns
# Index(['A', 'B', 'C', 'D'], dtype='object')

df.to_numpy() # export dataframe to numpy.ndarray
df.describe() # export mean, std, min, metrics for each column
df.mean() # computes mean
df.dropna() # drop NaN (not a number)
df.fillna(value=1) # fill entries with specific value
```

13

Accessing data

```
df["A"] # returns Series (column A)

df[0:2] # returns the first 2 rows

# boolean indexing
mask = df["A"] > 1
filtered_df = df[mask]
```

14

Scipy basics

Non-linear least squares

```
from scipy.optimize import curve_fit

def myfunc(x, a, b, c):
    return a * np.exp(-b * x) + c

xdata = np.load("x.dat")
ydata = np.load("y.dat")

popt, pcov = curve_fit(myfunc, xdata, ydata)
# popt: optimal values for the parameters
# pcov: estimated covariance of popt
```

↑ di. parametri

Usa i minimi quadrati per fare la regressione

15

Hessian approach

```
from scipy.optimize import minimize

xdata = np.load("x.dat")
ydata = np.load("y.dat")

def myfunc(x, a, b, c):
    return a * np.exp(-b * x) + c

def myloss(x):
    return np.sum(np.square(myfunc(xdata, x[0], x[1], x[2]) - ydata))

res = minimize(myloss, x0, method="BFGS", options={"disp": True})
# res.x: the solution of the optimization
```

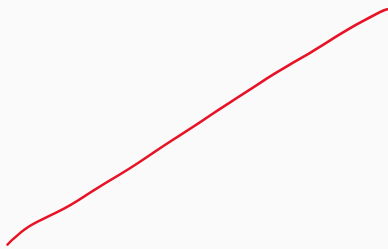
quasi-Newton

16

TensorFlow basics

TensorFlow

TensorFlow is a library for high performance numerical computation.



17

TensorFlow

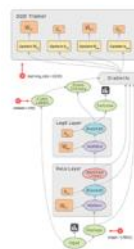
TensorFlow is a library for high performance numerical computation.

Pros:

- solves optimization problems with automatic differentiation.
- can be extended in python and c/c++.
- runs seamlessly on CPU and GPU, and can uses JIT technology.

Cons:

- do not provides builtin models from the core framework
- less automation for cross-validation and hyperparameter tune



Tensorflow crea subito il grafico, e cerca di parallelizzarlo in automatico, ottimizzando scegliendo gli hardware

Non ha molti modelli pronti, e neanche modelli di ottimizzazione hyperparametri

17

Tensors

```
import tensorflow as tf

x = tf.constant([1, 2, 3]) # immutable object

x.shape
x.dtype
x ** 2
tf.reduce_sum(x)

# similarly to numpy
a = tf.zeros(5, dtype=tf.float32)
b = tf.ones(5, dtype=np.float32)

# convert to numpy
c = b.numpy()
```

18

In questo caso tf usa multithreading or multicuda per riempire lo spazio che alloca. Usa quindi tutta la cpu o gpu, molto efficiente. Invece numpy è sequenziale e quindi più lento

Variables

```
import tensorflow as tf

var = tf.Variable([0, 0, 0]) # mutable object
var.assign([1, 1, 1])
```

19

Automatic differentiation

```
import tensorflow as tf

var = tf.Variable(1.0) # mutable object

def myfunc(x):
    return x ** 2 + 2 * x - 5

v = f(x) # <tf.Tensor: shape=(), dtype=float32, numpy=-2.0>

with tf.GradientTape() as tape: # gradient calculation
    y = f(x)

g_x = tape.gradient(y, x) # g(x) = dy/dx = (2 * x + 2)
g_x # <tf.Tensor: shape=(), dtype=float32, numpy=4.0>
```

20

uh? simbolico!

Graphs - performance optimization

```
import tensorflow as tf

@tf.function
def myfunc(x):
    print("Tracing")
    return tf.reduce_sum(x)

myfunc(tf.constant([1, 2, 3]))
# Tracing
# <tf.Tensor: shape=(), dtype=int32, numpy=6>

myfunc(tf.constant([-1, 2, -1]))
# <tf.Tensor: shape=(), dtype=int32, numpy=0>

myfunc(tf.constant([10.0, 9.1, 8.2], dtype=tf.float32))
# Tracing -> <tf.Tensor: shape=(), dtype=float32, numpy=27.3>
```

21

Just in time compiler

le faccio
sto questo esegue
solo il reduce_sum

se non lo
decoratore

stesso tipo di input

diverso

Boat
di efficienza

riesegue

la funzione prima è preservata?

Gli dico che voglio compilare questa funzione, quindi da questa riga in poi la funzione diventa un oggetto nativo di tf.

Se la chiamo passando una costante la esegue cercando di capire cosa fa (stampa Tracing). Fatto questo ogni chiamata dopo, ha già compilato la funzione,

Deep Learning - lab 3

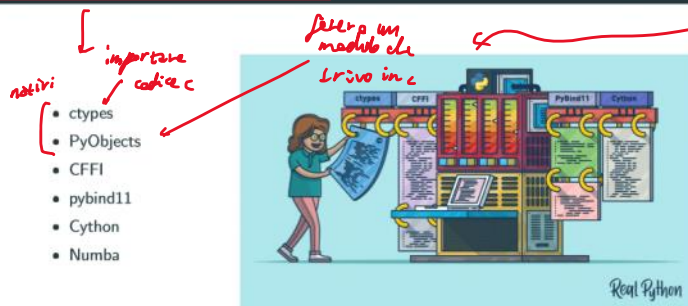
TensorFlow and Keras

Prof. Stefano Carrazza

University of Milan and INFN Milan

Extending Python

Extending Python with compilation



CFFI semplifica ctypes e PyObjects

Cython è un altro linguaggio

Numba è la più interessante, compilatore just in time. Basta mettere il decoratore @njit. Posso fare anche multithreading.

Numba sa compilare bene i tipi di dati di python. Cose più complicate potrebbe non ottenere neanche a compilare

Extending Python with compilation

Does deep learning require low level programming knowledge?

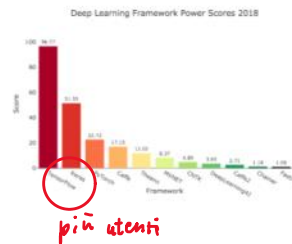
Depends:

- No: we can use frameworks based on compiled code.
- Yes: we can extend frameworks with custom operations (operators).

Which DL framework?

Make a decision based on:

- learning curve
- development pace
- community size
- papers implemented in the framework
- stability and long-term growth
- performance for specific task and hardware setup



Popolare per usare modelli conosciuti.

Se devo fare modelli nuovi particolari pytorch può essere meglio

3

TF variables and gradients

Placing variables and tensors

```
import tensorflow as tf

with tf.device('/CPU:0'): # places tensors and execute on CPU memory
    a = tf.Variable([[2.0, -3.0, 1.0], [1.0, 5.0, 3.0]])
    b = tf.constant([[1.0, 1.0], [2.0, 2.0], [3.0, 3.0]])
    c = tf.matmul(a, b)
print(c)

"""Output
tf.Tensor(
  [[-1. -1.]
   [20. 20.]], shape=(2, 2), dtype=float32)
"""
```

Posso dire quale device usare, CPU, GPU, ...

4

Placing variables and tensors

```
import tensorflow as tf

with tf.device('/CPU:0'): # places tensors and execute on CPU memory
    a = tf.Variable([[2.0, -3.0, 1.0], [1.0, 5.0, 3.0]])
    b = tf.constant([[1.0, 1.0], [2.0, 2.0], [3.0, 3.0]])
    with tf.device('/GPU:0'): # objects and operations move to GPU
        c = tf.matmul(a, b)
print(c)

"""Output
tf.Tensor(
  [[-1. -1.]
   [20. 20.]], shape=(2, 2), dtype=float32)
"""
```

Handwritten notes:

- ↳ copiato in GPU (pointing to the inner with block)
- ← matmul GPU (pointing to the matmul operation)
- ← copiato in automatico nella GPU (pointing to the inner with block)
- ↓ opie implicite (pointing to the inner with block)

5

TF gradients

Automatic gradient

```
import tensorflow as tf

x = tf.Variable(3.0)  # ← posso differenziare

with tf.GradientTape() as tape:
    y = x ** 2 + x

# dy/dx = 2x + 1
dy_dx = tape.gradient(y, x)
print(dy_dx.numpy())
```

6

Automatic gradient

```
import tensorflow as tf

w = tf.Variable(tf.random.normal((3, 2)), name='w')
b = tf.Variable(tf.zeros(2), dtype=tf.float32, name='b')
x = [[1., 2., 3.]] # automatic cast to tf.constant  ← bias
# di default è una costante

with tf.GradientTape() as tape:
    y = tf.nn.sigmoid(x @ w + b)  ← nomi per stampare
    loss = tf.reduce_mean(tf.math.square(y))  ←  $\sum \frac{1}{n} y_i^2$ 

[dl_dw, dl_db] = tape.gradient(loss, [w, b])
print(dl_dw, dl_db)
```

7

Automatic gradient

```
x0 = tf.Variable(3.0, name='x0') # A trainable variable
x1 = tf.Variable(3.0, name='x1', trainable=False) # Not trainable
x2 = tf.Variable(2.0, name='x2') 1.0 # Not a Variable: + tensor = a tensor.
x3 = tf.constant(3.0, name='x3') # Not a variable  ← costante!

with tf.GradientTape() as tape:
    y = (x0**2) + (x1**2) + (x2**2)  ← rispetto a

grad = tape.gradient(y, [x0, x1, x2, x3])
"""
tf.Tensor(6.0, shape=(), dtype=float32)
None
None
None
"""  ← costanti
```

8

TensorFlow Module

Custom Module

```
class SimpleModule(tf.Module):
    def __init__(self, name=None):
        super().__init__(name=name)
        self.a_variable = tf.Variable(5.0, name="train_me")
        self.non_trainable_variable = tf.Variable(5.0, trainable=False)

    def __call__(self, x):
        return self.a_variable * x + self.non_trainable_variable

simple_module = SimpleModule(name="simple")

simple_module(tf.constant(5.0))
```

uh? perché?

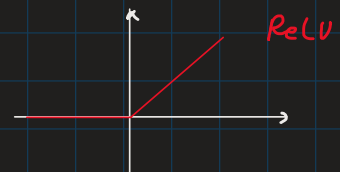
9

Custom Module

```
class Dense(tf.Module):
    def __init__(self, in_features, out_features, name=None):
        super().__init__(name=name)
        self.w = tf.Variable(
            tf.random.normal([in_features, out_features]), name='w')
        self.b = tf.Variable(tf.zeros([out_features]), name='b')

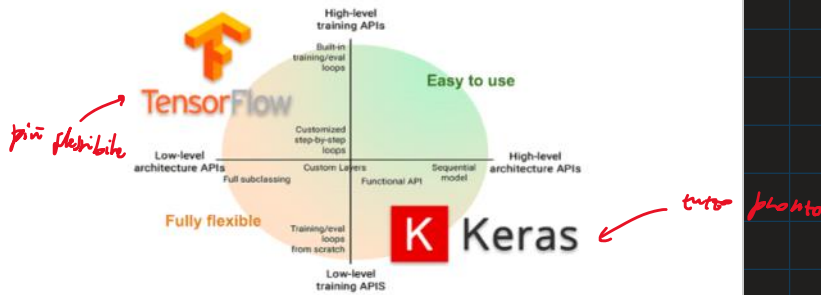
    def __call__(self, x):
        y = tf.matmul(x, self.w) + self.b
        return tf.nn.relu(y)
```

10



Keras

Keras



11

Sequential model

```
import tensorflow as tf
model = tf.keras.Sequential()
model.add(tf.keras.layers.Dense(2, activation="relu"))
model.add(tf.keras.layers.Dense(3, activation="relu"))
model.add(tf.keras.layers.Dense(1))

# call model
y = model(tf.ones((1,1)))
"""
<tf.Tensor: shape=(1, 1), dtype=float32,
  numpy=array([[0.09049725]], dtype=float32)>
"""
```



~ lineare

12

Sequential model

```
# print model summary
model.summary()
"""
Model: "sequential"

Layer (type)                 Output Shape         Param #
=====
dense (Dense)                (None, 2)            4
dense_1 (Dense)              (None, 3)            9
dense_2 (Dense)              (None, 1)            4
=====
Total params: 17
Trainable params: 17
Non-trainable params: 0
"""
```

13

Sequential model

```
import tensorflow as tf
model = tf.keras.Sequential()
model.add(...)

# compile model
model.compile(...)

# train model
model.fit(...)
```

Handwritten notes:

- loss
- ottimizzatore
- metriche
- dati

14

Sequential model

va bene per case base

A Sequential model is not appropriate when:

- Your model has multiple inputs or multiple outputs
- Any of your layers has multiple inputs or multiple outputs
- You need to do layer sharing
- You want non-linear topology (e.g. a residual connection, a multi-branch model)

Solution → use the functional API

15

The Functional API

lo costruisce a mano

```
import tensorflow as tf
```

calcolo sequenziale

```
# build model step by step
```

```
inputs = tf.keras.Input(shape=(5,))
```

```
x1 = tf.keras.layers.Dense(64, activation="relu")(inputs)
```

```
x2 = tf.keras.layers.Dense(64, activation="relu")(x1)
```

```
outputs = tf.keras.layers.Dense(1)(x2)
```

```
# build model from functional API
```

```
model = tf.keras.Model(inputs=inputs, outputs=outputs, name="mymodel")
```

16

The Functional API

```
import tensorflow as tf
```

```
# ...
```

```
# load model and data
```

```
# ...
```

```
model = tf.keras.Model(inputs=inputs, outputs=outputs, name="mymodel")
```

```
# compile model with custom loss and optimizer
```

```
model.compile(loss="mean_squared_error",  
              optimizer=tf.keras.optimizers.SGD())
```

```
# train and save history
```

```
history = model.fit(x_train, y_train, batch_size=64, epochs=1000)
```

```
# evaluate model performance on separate test set
```

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

17

Deep Learning - lab 4

Hardware acceleration, keras, hyperparametrization

Prof. Stefano Carrazza

University of Milan and INFN Milan

Hardware acceleration

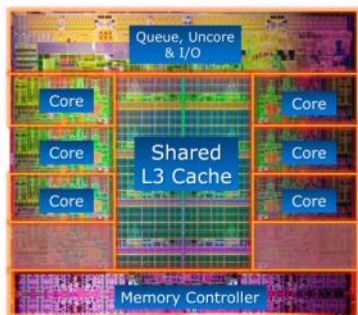
CPU threading



1

Multithreading sulle CPU

CPU threading



2

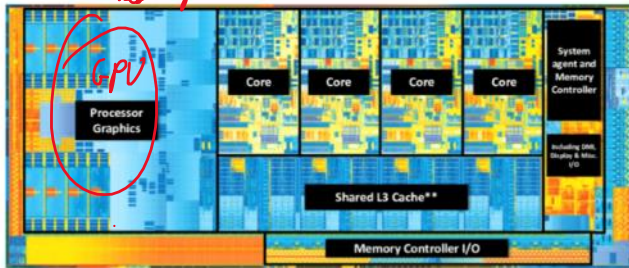
I core fanno calcoli in modo asincrono, indipendente.

Se il codice è parallelizzabile posso utilizzarli insieme.

Il problema è la memoria cache quando si multithreada

CPU threading

non più thread ma più lentamente come clock speed



Each CPU core (in a Multi-core CPU) can have multi-thread support (usually 2 threads per core) using hyper-threading technology.

3

CPU threading

Modern CPUs support three types of parallelism:

- instruction-level parallelism (ILP), done automatically by CPU.
- single instruction, multiple data (SIMD) instructions (SSE, AVX2, etc.)
- thread-level parallelism (TLP) → using compilers/libraries

4

CPU threading

Single-threaded code:

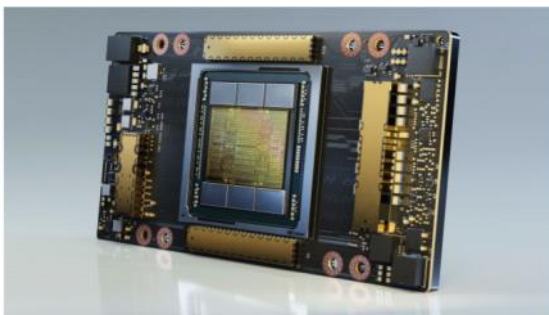
```
const int N = 100000;
double sum = 0, a[N];
for (int i = 0; i < N; i++)
    a[i] = 2 * i;
```

Multi-threading code (using openMP):

```
const int N = 100000;
double sum = 0, a[N];
#pragma omp parallel for
for (int i = 0; i < N; i++)
    a[i] = 2 * i;
```

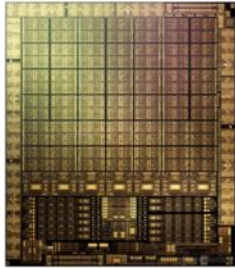
5

GPU architecture



6

GPU architecture



- Larger number of threads when compared to CPU
- High power consumption
- Slower clock speed in comparison to CPU
- Limited RAM memory, e.g. 80GB maximum
- Requires special compilers and SDK, e.g. CUDA (NVIDIA) or ROCm (AMD)

7

FPGA

Stampo calcoli sul chip, posso riscrivere l'architettura ai per esempio sul chip.

Ad esempio tramite fpga pierini hls4ml(keras->converte in un firmware).

Consumo molto basso energetico, e molto veloce.

Lo step successivo è l'ASIC che non è riprogrammabile come le FPGA

Le TPU sono un tipo di FPGA(allenamento e inferenza efficiente)

Keras built-in methods

Keras

```
inputs = keras.Input(shape=(784,), name="digits")
x = layers.Dense(64, activation="relu", name="dense_1")(inputs)
x = layers.Dense(64, activation="relu", name="dense_2")(x)
outputs = layers.Dense(10, activation="softmax", name="predictions")(x)

model = keras.Model(inputs=inputs, outputs=outputs)

model.compile(
    optimizer=keras.optimizers.RMSprop(), # Optimizer
    # Loss function to minimize
    loss=keras.losses.SparseCategoricalCrossentropy(),
    # List of metrics to monitor
    metrics=[keras.metrics.SparseCategoricalAccuracy()],
)
```

8

Per modelli più complessi

Keras

```
inputs = keras.Input(shape=(784,), name="digits")
x = layers.Dense(64, activation="relu", name="dense_1")(inputs)
x = layers.Dense(64, activation="relu", name="dense_2")(x)
outputs = layers.Dense(10, activation="softmax", name="predictions")(x)

model = keras.Model(inputs=inputs, outputs=outputs)

model.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy",
    metrics=["sparse_categorical_accuracy"],
)
```

9

Keras

```
history = model.fit(
    x_train,
    y_train,
    batch_size=64,
    epochs=2,
    # We pass some validation for monitoring validation loss and metrics
    # at the end of each epoch
    validation_data=(x_val, y_val),
)
print(history.history)
"""
{'loss': [0.3386789858341217, 0.1543138176202774],
'sparse_categorical_accuracy': [0.9050400257110596, 0.954840004441223],
'val_loss': [0.19569723308086395, 0.14253544807434082],
'val_sparse_categorical_accuracy': [0.9426000118255615, 0.9592999815940857]}
"""
```

10

Keras

```
# Evaluate the model on the test data using `evaluate`
print("Evaluate on test data")
results = model.evaluate(x_test, y_test, batch_size=128)
print("test loss, test acc:", results)

# Generate predictions (probabilities -- the output of the last layer)
# on new data using `predict`
print("Generate predictions for 3 samples")
predictions = model.predict(x_test[:3])
print("predictions shape:", predictions.shape)
```

11

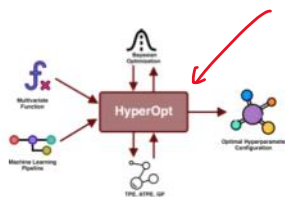
Hyperparametrization

Hyperopt

Hyperopt is a simple python package which provides:

- Random Search
- Tree of Parzen Estimators (TPE)
- Adaptive TPE

libreria da usare



<https://github.com/hyperopt>

12

HyperOpt è quello originale più vecchio

Optuna è più recente, anche carina

Hyperopt expressions

```
from hyperopt import hp
```

```
# define search space
```

```
a = hp.uniform('a', -10, 10)
```

```
b = hp.choice('b', [1, 2, 3, 4])
```

```
c = hp.loguniform('c', -5, 0)
```

```
# randint, normal, lognormal, ...
```

definisce lo spazio dei parametri

learning rate

13

Hyperopt basics

```
from hyperopt import fmin, tpe, hp
```

```
# define search space
```

```
space = hp.uniform('x', -10, 10)
```

```
# function to be minimized
```

```
def objective(x):
```

```
    return x ** 2
```

```
best = fmin(objective, space, algo=tpe.suggest, max_evals=100)
```

```
print(best)
```

```
"""{'x': 0.03866588214338945}"""
```

punto ottimale

14

Hyperopt basics

```
from hyperopt import fmin, tpe, hp, space_eval
```

```
# define search space
```

```
space = hp.uniform('x', -10, 10)
```

```
# function to be minimized
```

```
def objective(x):
```

```
    return x ** 2
```

```
best = fmin(objective, space, algo=tpe.suggest, max_evals=100)
```

```
print(best)
```

```
"""{'x': 0.03866588214338945}"""
```

```
print(space_eval(space, best))
```

```
"""0.03866588214338945"""
```

valore della parabola

15

Hyperopt attaching extra information

```
from hyperopt import fmin, tpe, hp, STATUS_OK
```

```
# define search space
```

```
space = hp.uniform('x', -10, 10)
```

```
# function to be minimized
```

```
def objective(x):
```

```
    return {'loss': x ** 2, 'status': STATUS_OK}
```

```
best = fmin(objective, space, algo=tpe.suggest, max_evals=100)
```

```
print(best)
```

```
"""{'x': 0.03866588214338945}"""
```

OK, K6

Utile per escludere per esempio modelli che non convergono o che non sono fisici o non hanno senso e voglio escluderle. Posso avvertire hyperopt che la cosa è andata male.

Se voglio massimizzare metto un -

16

Hyperopt the trials object

```
import time
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials

def objective(x):
    return {'loss': x ** 2, 'status': STATUS_OK, 'eval_time': time.time()}

trials = Trials() # objecting collecting sequential trials
best = fmin(objective, space=hp.uniform('x', -10, 10),
            algo=tpe.suggest, max_evals=100,
            trials=trials)

# trials.trials      : list of dicts representing everything about the search
# trials.results     : list of dicts returned by objective during the search
# trials.losses()    : list of losses (matching ok)
# trials.statuses() : list of status strings
```

17

Posso salvare informazioni

Dà l'history con tutti gli obiettivi

Deep Learning - lab 5

Hyperopt and RNNs

Prof. Stefano Carrazza

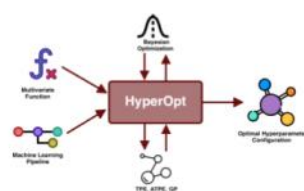
University of Milan and INFN Milan

Hyperparametrization

Hyperopt

Hyperopt is a simple python package which provides:

- Random Search
- Tree of Parzen Estimators (TPE)
- Adaptive TPE



<https://github.com/hyperopt>

1

Hyperopt expressions

```

from hyperopt import hp, pyll

# define search space
a = hp.uniform('a', -10, 10)
b = hp.choice('b', [1, 2, 3, 4])
c = hp.loguniform('c', -5, 0)
# randint, normal, lognormal, ...

sample = pyll.stochastic.sample(a) # generate 1 sample
    
```

*vedere il
campionamento*

2

Hyperopt the trials object

```
import time
from hyperopt import fmin, tpe, rand, hp, STATUS_OK, Trials

def objective(x):
    return {'loss': x ** 2, 'status': STATUS_OK, 'eval_time': time.time()}

algorithm = tpe.suggest # or rand.suggest
trials = Trials() # objecting collecting sequential trials
best = fmin(objective, space=hp.uniform('x', -10, 10),
            algo=algorithm, max_evals=100,
            trials=trials)

print(best) # returns dict with {'x': value}
iterations = trials.idx_vals[0]['x']
x_values = trials.idx_vals[1]['x']
```

3

Hyperopt the trials object

```
from hyperopt import fmin, tpe, hp, Trials, space_eval, STATUS_OK

search_space = {
    'layer_size': hp.choice('layer_size', np.arange(10, 100, 20)),
    'learning_rate': hp.loguniform('learning_rate', -10, 0)
}

trials = Trials()
best = fmin(hyper_func, search_space, algo=tpe.suggest,
            max_evals=5, trials=trials)

space_eval(search_space, best) # translates dict to real search space
value
```

4

Trials

```
trials = Trials()

# Extracting history
iterations = [ t['tid'] for t in trials.trials]
losses = [ t['result']['loss'] for t in trials.trials]
learning_rates = [ t['misc']['vals']['learning_rate'] for t in trials.trials]
```

5

RNNs

Built-in RNN models

Keras provides 3 built-in RNN layers:

- `tensorflow.keras.layers.SimpleRNN`: fully-connected RNN.
- `tensorflow.keras.layers.LSTM`
- `tensorflow.keras.layers.GRU`

and there are three built-in RNN cells, matching the layers:

- `tensorflow.keras.layers.SimpleRNNCell`
- `tensorflow.keras.layers.GRUCell`
- `tensorflow.keras.layers.LSTMCell`

6

A simple LSTM regression

```
from tensorflow import keras
```

```
model = keras.Sequential()
```

```
# suppose
```

```
model.add(keras.layers.Input(shape=(32,1)))
```

```
# add a LSTM layer with 10 internal units
```

```
model.add(keras.layers.LSTM(10))
```

```
# add a dense layer with 1 unit
```

```
model.add(keras.layers.Dense(1))
```

7

A simple GRU regression

By default the output of a RNN layer is a single vector per sample.

```
from tensorflow import keras
```

```
model = keras.Sequential()
```

```
# suppose
```

```
model.add(keras.layers.Input(shape=(32,1)))
```

```
# add a GRU layer with 10 internal units
```

```
# returns (batch_size, timesteps, 10)
```

```
model.add(keras.layers.GRU(10, return_sequences=True))
```

```
# add a dense layer with 1 unit
```

```
model.add(keras.layers.Dense(1))
```

8

Encoder-decoder sequence-to-sequence

Using the functional API we can design more complex models:

```
encoder_input = keras.layers.Input(shape=(None,1))
```

```
# return states in addition to output
```

```
output, state_h, state_c = layers.LSTM(64, return_state=True)(encoder_input)
```

```
encoder_state = [state_h, state_c]
```

```
decoder_input = layers.Input(shape=(None,1))
```

```
decoder_output = layers.LSTM(64)(decoder_input, initial_state=encoder_state)
```

```
output = layers.Dense(10)(decoder_output)
```

```
model = keras.Model([encoder_input, decoder_input], output)
```

9

Bidirectional

```
model = keras.Sequential()

model.add(layers.Bidirectional(layers.LSTM(64, return_sequences=True),
                             input_shape=(5, 10)))

model.add(layers.Bidirectional(layers.LSTM(32)))
model.add(layers.Dense(10))
```

Deep Learning - lab 6

Keras callbacks and CNNs

Prof. Stefano Carrazza

University of Milan and INFN Milan

Keras Callbacks

Utile per fare operazioni durante il training tipo early stops

O anche per fare checkpoints con cui salvo il modello, e con tf tensorboard posso vedere statistiche dei checkpoint

Keras callbacks

A callback is an object that can perform actions at various stages of training (e.g. at the start or end of an epoch, before or after a single batch, etc).

You can use callbacks to:

- Periodically save your model to disk
- Do early stopping
- Write TensorBoard logs after every batch of training to monitor your metrics
- Get a view on internal states and statistics of a model during training
- ...and more

1

Keras callback API

Example: storing model parameters during training

```
# save the weights manually
model = create_model()
model.fit(...)
model.save_weights('./checkpoints/my_checkpoint')

# restore weights manually
model = create_model()
model.load_weights('./checkpoints/my_checkpoint')
```

2

Salvare e caricare pesi(prima o dopo l'allenamento)

Keras callback API

Example: storing model parameters during training

```
from tensorflow import keras

model = ... # define your model as usual

# declare callbacks
my_callbacks = [
    keras.callbacks.ModelCheckpoint(filepath='model.{epoch}-{val_loss:.2f}.h5'),
]

model.fit(data, epochs=10, callbacks=my_callbacks)
```

3

Se voglio salvare all'interno dell'allenamento posso usare i callbacks

Keras callback API

ModelCheckpoint

ModelCheckpoint class

```
tf.keras.callbacks.ModelCheckpoint(
    filepath,
    monitor="val_loss",
    verbose=0,
    save_best_only=False,
    save_weights_only=False,
    mode="auto",
    save_freq="epoch",
    options=None,
    initial_value_threshold=None,
    **kwargs
)
```

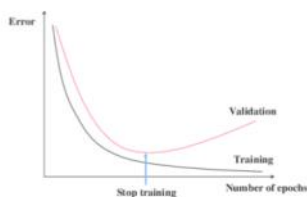
Callback to save the Keras model or model weights at some frequency.

ModelCheckpoint callback is used in conjunction with training using `model.fit()` to save a model or weights (in a checkpoint file) at some interval, so the model or weights can be loaded later to continue the training from the state saved.

4

Keras callback API

Example: early stopping



```
from tensorflow import keras

model = ... # define your model

# declare callbacks
my_callbacks = [
    keras.callbacks.EarlyStopping(patience=2),
]

model.fit(data, epochs=10,
          callbacks=my_callbacks)
```

Utile per l'early stopping

Epochs da aspettare prima di fermarmi quando le cose iniziano ad andare male

5

Keras callback API

EarlyStopping

EarlyStopping class

```
tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    min_delta=0,
    patience=0,
    verbose=0,
    mode="auto",
    baseline=None,
    restore_best_weights=False,
)
```

media min delta 0 -> 1000

Stop training when a monitored metric has stopped improving.

Assuming the goal of a training is to minimize the loss. With this, the metric to be monitored would be 'loss', and mode would be 'min'. A `model.fit()` training loop will check at end of every epoch whether the loss is no longer decreasing, considering the `min_delta` and `patience` if applicable. Once it's found no longer decreasing, `model.stop_training` is marked True and the training terminates.

6

Keras callback API

TensorBoard provides the visualization and tooling needed for machine learning experimentation:

- Tracking and visualizing metrics such as loss and accuracy
- Visualizing the model graph
- Viewing histograms of weights, biases, or other tensors as they change over time
- Projecting embeddings to a lower dimensional space
- Displaying images, text, and audio data
- Profiling TensorFlow programs



7

Keras callback API

Example: storing logs for TensorBoard

```
from tensorflow import keras

model = ... # define your model

# declare callbacks
my_callbacks = [
    keras.callbacks.TensorBoard(log_dir='./logs'),
    keras.callbacks.EarlyStopping(patience=2),
]

model.fit(data, epochs=10, callbacks=my_callbacks)

# open tensorboard with
# tensorboard --logdir=path_to_your_logs
```

← tensorboard

8

Keras callback API

TensorBoard

TensorBoard class

```
if keras.callbacks.TensorBoard:
    log_dir='./logs',
    histogram_freq=0,
    write_graph=True,
    write_images=False,
    write_steps_per_second=False,
    update_freq='epoch',
    profile_batch=0,
    embeddings_freq=0,
    embeddings_metadata=None,
    'twarg'
```

Enable visualizations for TensorBoard.

TensorBoard is a visualization tool provided with TensorFlow.

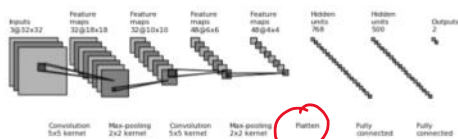
This callback logs events for TensorBoard, including:

- Metrics summary plots
- Training graph visualization
- Weight histograms
- Sampled profiling

9

Convolutional Neural Nets

Convolutional base



```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(128))
model.add(layers.Dense(2))
```

10

Conv2D

Conv2D layer

Conv2D class

```
tf.keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    data_format="channels_last",
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

2D convolution layer (e.g. spatial convolution over images).

This layer creates a convolution kernel that is convolved with the layer input to produce a tensor of outputs. If `use_bias` is True, a bias vector is created and added to the outputs. Finally, if `activation` is not None, it is applied to the outputs as well.

11

MaxPooling2D

MaxPooling2D layer

MaxPooling2D class

```
tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2), strides=None, padding="valid", data_format="channels_last",
    **kwargs
)
```

Max pooling operation for 2D spatial data.

Downsamples the input along its spatial dimensions (height and width) by taking the maximum value over an input window (of size defined by `pool_size`) for each channel of the input. The window is shifted by `strides` along each dimension.

The resulting output, when using the "valid" padding option, has a spatial shape (number of rows or columns) of: `output_shape = math.floor((input_shape - pool_size) / strides) + 1` (when `input_shape >= pool_size`)

The resulting output shape when using the "same" padding option is: `output_shape = math.floor((input_shape + 1) / strides) + 1`

12

Example: CNN classifier

```
num_classes = len(class_names)

model = Sequential([
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])
```

13

Deep Learning - lab 7

Object detection

Prof. Stefano Carrazza

University of Milan and INFN Milan

Classification with localization

Keras

```
import tensorflow as tf

inputs = tf.keras.layers.Input(shape=(28, 28, 1))
c1 = tf.keras.layers.Conv2D(64, 3, activation='relu')(inputs)
c2 = tf.keras.layers.MaxPooling2D((3, 3))(c1)
...
cN = tf.keras.layers.Flatten()(cN_1)
x = tf.keras.layers.Dense(128, activation='relu')(cN)

# output for classifier
o1 = tf.keras.layers.Dense(10, activation='softmax', name='classifier')(x)
# output for bbox regressor
o2 = tf.keras.layers.Dense(4, name='regressor')(x)

model = tf.keras.Model(inputs=inputs, outputs=[o1, o2])
```

1

Keras

```
model.compile(optimizer='adam',
              loss={
                  'classifier': 'categorical_crossentropy',
                  'regressor': 'mse'
              },
              metrics=[
                  'classifier': 'acc',
                  'regressor': 'mse'
              ])

model.fit(train_images, (train_labels, train_bbox), ...)
```

2

Predico 2 valori diversi, come costruirlo

CNN

classificatore normale
bounding box

2 tipo di output

i nomi con importanti

Lo alleno, usando loss functions e metriche per i diversi pezzi del modello

Object detection

Tensorflow

 **TensorFlow Model Garden**

Welcome to the Model Garden for TensorFlow

The TensorFlow Model Garden is a repository with a number of different implementations of state-of-the-art (SOTA) models and modeling solutions for TensorFlow users. We aim to demonstrate the best practices for modeling so that TensorFlow users can take full advantage of TensorFlow for their research and product development.

To improve the transparency and reproducibility of our models, training logs on [TensorBoard](#) are also provided for models to the extent possible though not all models are suitable.

Directory	Description
official	- A collection of example implementations for SOTA models using the latest TensorFlow 2's high-level APIs. - Officially maintained, supported, and kept up to date with the latest TensorFlow 2 APIs by TensorFlow. - Highly optimized for fast performance while still being easy to read.
research	- A collection of research model implementations in TensorFlow 1 or 2 by researchers. - Maintained and supported by researchers.
community	A curated list of the GitHub repositories with machine learning models and implementations powered by TensorFlow 2.
util	A flexible and lightweight library that users can easily use or fork when writing customized training loop code in TensorFlow 2.x. It seamlessly integrates with <code>tf.nn.parallel</code> and supports running on different device types (CPU, GPU, and TPU).

<https://github.com/tensorflow/models>

Model garden ha molti modelli facili da integrare

Detectron2

 **Detectron2**

pytorch

Detectron2 is a state-of-the-art object detection and segmentation library that provides state-of-the-art detection and segmentation algorithms. It is the successor of Detectron and m2cnn. It supports a number of computer vision research projects and production applications in Facebook.



<https://github.com/facebookresearch/detectron2>

MMDetection

 **MMDetection**

OpenMMLab website HOT OpenMMLab platform TRY IT OUT

[v2.13.0](#) [Docs](#) [Release](#) [PyTorch](#) [Testing](#) [Codebase](#) [C++](#) [License](#) [License-CC](#) [Report Issues](#) [C2C](#)



[Documentation](#) | [Installation](#) | [Model Zoo](#) | [Update News](#) | [Ongoing Projects](#) | [Reporting Issues](#)

<https://github.com/open-mmlab/mmdetection>

Deep Learning - lab 8

Data augmentation and transfer learning

Prof. Stefano Carrazza

University of Milan and INFN Milan

Loading data from files

Keras get file

```
import tensorflow as tf

# example tgz file from:
dataset_url = 'https://somelocation/../../data.tgz'
data_dir = tf.keras.utils.get_file('save_location',
                                   origin=dataset_url,
                                   extract=True)
```

1

Get_file scarica i dati in una cartella e fa estrazioni in automatico.
Utile se c'è un server con dati o roba così.

Keras get file

```
import tensorflow as tf

# example zip file from:
dataset_url = 'https://somelocation/../../data.zip'
data_dir = tf.keras.utils.get_file('data_features.zip',
                                   origin=dataset_url,
                                   extract=True)
```

2

Keras get file

Suppose your data directory structure is:

```
main_directory/  
...class_a/  
.....a_image_1.jpg  
.....a_image_2.jpg  
...class_b/  
.....b_image_1.jpg  
.....b_image_2.jpg
```

3

Se lo faccio la struttura dei dati deve essere fatta così

Keras image dataset from directory

```
train_ds = tf.keras.utils.image_dataset_from_directory(  
    data_dir,  
    validation_split=0.2,  
    subset="training",  
    image_size=(img_height, img_width),  
    batch_size=batch_size)  
  
# extract class names  
class_names = train_ds.class_names  
  
# train_ds is a tf.Dataset
```

4

Una volta caricato il dataset con questa tecnica, ho il vantaggio che dall'utils posso manipolare i dati per esempio così.

Un modo compatto di scrivere tutte le proprietà del dataset in anteprima

Keras image dataset from directory

```
val_ds = tf.keras.utils.image_dataset_from_directory(  
    data_dir,  
    validation_split=0.2,  
    subset="validation",  
    image_size=(img_height, img_width),  
    batch_size=batch_size)
```

5

tf.data.Dataset

```
# creates dataset with at most 1 element  
train_ds.take(1) ← un elemento console, vedo un campione  
  
# keeps the images in memory after they're loaded off disk at epoch=1  
# if dataset is too large to fit into memory, this method can create a  
# performance on-disk cache.  
train_ds.cache()  
  
# Randomly shuffles the elements of this dataset.  
train_ds.cache().shuffle(1000)  
  
# overlaps data preprocessing and model execution while training  
train_ds.cache().prefetch(tf.data.AUTOTUNE)
```

6

Vantaggi dell'approccio

Questo è pensato per dataset pesanti. Se devo caricare queste immagini su una memoria, il .cache() carica le immagini in memoria, e poi cerca a tutti i costi di tenerle in memoria, in modo che è più veloce. A meno che la memoria è più piccola del dataset, in quel caso crea su disco cose, e la memoria dinamicamente accede ai vari così. Nel caso delle GPU conviene. Il memory manager dice quale pezzi pulire e ricaricare da disco. Usando questo i dati saranno in continuazione caricati e scaricati in modo asincrono rispetto l'allenamento. Molto efficiente

In fase di esecuzione analizza come i dati sono usati e fixa roba, in modo da usare in modo più ottimale il coso di memoria.

Tutto questo ovviamente se il dataset occupa più della memoria, altrimenti rimane lì fermo e bona.

Data augmentation

Data augmentation

l'allenamento. Molto efficiente

In fase di esecuzione analizza come i dati sono usati e fixa roba, in modo da usare in modo più ottimale il coso di memoria.

Tutto questo ovviamente se il dataset occupa più della memoria, altrimenti rimane lì fermo e bona.

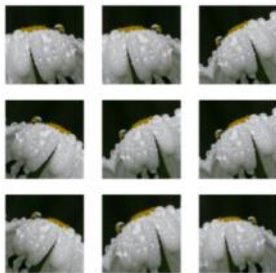
Builtin methods

```
data_augmentation = tf.keras.models.Sequential([
    tf.keras.layers.RandomFlip("horizontal",
                               input_shape=(img_height, img_width, 3)),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
])
```

7

Keras offre alcuni layer per data augmentation geometrico. Per esempio un flip orizzontale, rotazioni random, o flip random, per ogni immagine, per ogni caricamento, quindi ho sempre nuovi dati e il dataset si allarga abbastanza.

Builtin methods



8

Se aumento lo zoom si nota di più, qua è poco.

Quindi se fa fatica ad allenare posso provare ad usarlo

Transfer learning

Loading model

```
base_model = tf.keras.applications.MobileNetV2(input_shape=(IMG_SHAPE),  
                                                include_top=False,  
                                                weight='imagenet')
```

9

Infine, se voglio fare transfer learning, dal punto di vista pratico non c'è molto da sapere se non i nomi dei modelli. Per scaricarlo uso questo comando + qualche settaggio